

Statistical Foundations & Probability

Reasoning Under Uncertainty, Limit Laws, and Hypothesis Testing

TOPIC 01

The Illusion of Certainty: Bayes' Theorem

Rare events, base rate fallacy, conditional probability geometry, and expectation.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Deconstruct the Base Rate Fallacy** in clinical diagnostics, fraud detection, and rare-event anomaly modeling.
- **Partition probability sample spaces** geometrically using joint, marginal, and conditional frequency trees.
- **Track belief updates visually** via the $\text{Prior} \times \text{Likelihood} \rightarrow \text{Posterior}$ Bayesian engine.
- **Implement a Bayesian belief updater** in vectorized NumPy for real-time sequential telemetry.

The Rare Event Paradox

The Production Dilemma: Fraud Detection

- Base rate of fraudulent transactions: 1% ($P(F) = 0.01$).
- ML Classifier Accuracy: 95% Sensitivity, 95% Specificity (5% False Positive Rate).
- An alert fires on Transaction #84920!
- **Socratic Probe:** What is the probability that this transaction is **actually** fraudulent?
- **Intuitive Guess:** “95%!” → **Completely wrong!**

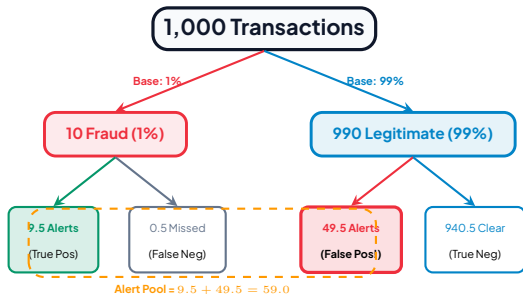
The “Aha!” Discovery: Base Rate Dominance

Consider 1,000 transactions:

- **10 Fraudulent:** 9.5 trigger True Alert, 0.5 Missed.
- **990 Legitimate:** 5% of 990 = **49.5** trigger **False Alerts!**
- Total Alerts Fired: $9.5 + 49.5 = 59.0$.
- True Probability: $\frac{9.5}{59.0} \approx 16.1\%$

Important: Base rates dominate high-accuracy test outcomes.

Hero Visualization: Frequency Tree Partitioning



Bayes' Theorem Formalized

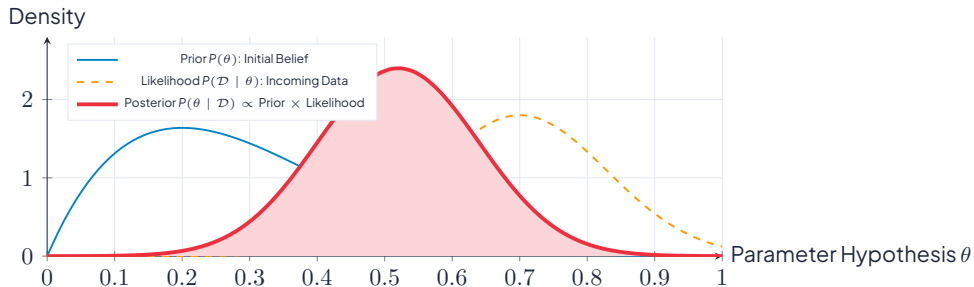
- Bayes' Formula:

$$P(F | +) = \frac{P(+ | F)P(F)}{P(+ | F)P(F) + P(+ | \neg F)P(\neg F)}$$

- Numerator (True Signal):
 $0.95 \times 0.01 = 0.0095$.
- Denominator (Total Evidence):
 $0.0095 + (0.05 \times 0.99) = 0.0590$.
- Posterior Probability:

$$P(F | +) = \frac{0.0095}{0.0590} = 16.1\%$$

Hero Visualization: Prior $\times$ Likelihood $\rightarrow$ Posterior



Important

Today's posterior becomes tomorrow's prior in recursive learning systems.

Production Checkpoint: Vectorized Bayesian Filter

Sequential Bayesian Inference

For hypothesis space $\Theta = [\theta_1, \dots, \theta_K]$:

- **Prior:** $p_0 \in \mathbb{R}^K$ (Uniform or historical).
- **Update Rule:**

$$p_t = \frac{p_{t-1} \odot L(\text{obs} \mid \Theta)}{\sum_{k=1}^K p_{t-1,k} L(\text{obs} \mid \theta_k)}$$

- Exact, closed-form updates without MCMC sampling for discrete parameter spaces.

Vectorized NumPy Implementation

```
1 import numpy as np
2
3 class DiscreteBayesianFilter:
4     def __init__(self, hypotheses: np.ndarray):
5         self.hypo = hypotheses
6         # Start with uniform prior
7         self.prior = np.ones_like(hypotheses) / len(hypotheses)
8
9     def update(self, likelihoods: np.ndarray) -> np.ndarray:
10        """likelihoods: P(Data | hypo_k)"""
11        unnormalized = self.prior * likelihoods
12        evidence = np.sum(unnormalized)
13        self.prior = unnormalized / evidence
14        return self.prior
```

TOPIC 02

Order from Chaos: The Central Limit Theorem

Why bell curves rule the universe, Law of Large Numbers, and standard errors.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Explain the universality of the Central Limit Theorem (CLT)** across arbitrary non-normal parent distributions.
- **Quantify sampling uncertainty** via the Standard Error formula $SE = \frac{\sigma}{\sqrt{N}}$ and variance collapse.
- **Navigate the Distribution Zoo** (Bernoulli, Poisson, Exponential, Normal) to select proper likelihood models for data.
- **Run high-speed Monte Carlo simulations** in NumPy to empirically validate limit laws.

The Variance Collapse Mystery

The Wild Individual vs The Calm Average

- Individual server response times $X_i \sim \text{Exponential}(\lambda)$ are heavily skewed with extreme latency spikes.
- Measure the mean latency $\bar{X}_N = \frac{1}{N} \sum_{i=1}^N X_i$ across batches of $N = 100$ requests.
- **Socratic Probe:** Why does the distribution of batch means become perfectly symmetrical and narrow?

The "Aha!" Discovery: $\frac{1}{\sqrt{N}}$ Scaling

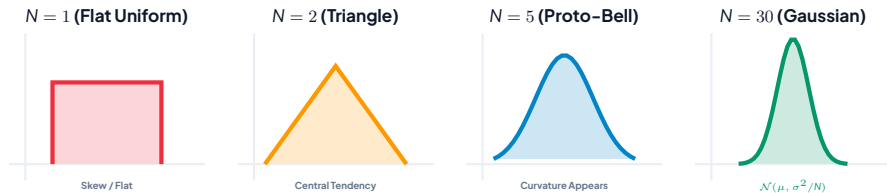
- **Expected Value:** $\mathbb{E}[\bar{X}_N] = \mu$ (Unbiased center).
- **Variance Collapse:**

$$\text{Var}(\bar{X}_N) = \text{Var}\left(\frac{1}{N} \sum X_i\right) = \frac{1}{N^2} \sum \sigma^2 = \frac{\sigma^2}{N}$$

- **Standard Error:** $\text{SE} = \frac{\sigma}{\sqrt{N}}$.
- Independent random fluctuations cancel each other out!

Important Quadrupling sample size cuts estimation error in half.

Hero Visualization: CLT Convergence from Uniform Noise



Important

Summing independent variables inevitably produces Gaussian distribution geometry.

Hero Visualization: The Distribution Selection Matrix

Discrete Counting Processes

- **Bernoulli(p):** Single binary coin flip / click conversion ($k \in \{0, 1\}$).
- **Binomial(n, p):** Number of successes in n independent trials ($k \in \{0, \dots, n\}$).
- **Poisson(λ):** Rare count events per fixed time window (e.g., website server errors/min). Mean = Variance = λ .

Continuous Physical Quantities

- **Uniform(a, b):** Maximum uncertainty / random initialization in neural nets.
- **Exponential(λ):** Waiting time between Poisson events (memoryless queue latency).
- **Gaussian $\mathcal{N}(\mu, \sigma^2)$:** Aggregated sensor noise, measurement errors, regression residuals.

Match your ML loss function to
the data generation
distribution.

Production Checkpoint: Empirical Monte Carlo CLT Engine

Monte Carlo Simulation Mechanics

- Draw $M = 100,000$ independent sample batches from heavy-tailed Pareto / Exponential distribution.
- Each batch contains $N = 50$ observations.
- Compute batch sample means across columns in one vectorized operation.
- Verify that kurtosis and skewness converge to Gaussian (0.0).

Vectorized NumPy Simulation

```
1 import numpy as np
2
3 def simulate_clt(
4     num_experiments=100_000,
5     sample_size=50
6 ) -> tuple[float, float]:
7     # 1. Draw from heavily-skewed Exponential
8     raw_samples = np.random.exponential(
9         scale=2.0,
10        size=(num_experiments, sample_size)
11    )
12    # 2. Vectorized batch means: (100000,)
13    sample_means = np.mean(raw_samples, axis=1)
14
15    # 3. Empirical Mean & Standard Error
16    return float(np.mean(sample_means)), float(np.std(sample_means))
```

TOPIC 03

Hypothesis Testing & Decision Making

The skeptic's null hypothesis, p-values, Type I/II errors, and statistical power.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Formulate testable hypotheses** with the Null Hypothesis (H_0) as the conservative presumption of zero effect.
- **Interpret p -values accurately** as the probability of observing data at least as extreme under H_0 (not the probability that H_0 is true).
- **Balance Type I (α) vs Type II (β) errors** to maximize statistical power ($1 - \beta$) in production A/B tests.
- **Execute non-parametric permutation tests** in NumPy to evaluate metrics without asymptotic normality assumptions.

The A/B Testing Dilemma

The Business Question: Real or Luck?

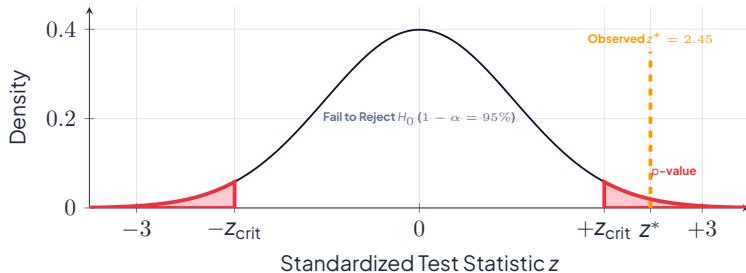
- Control (A): 2,000 users $\rightarrow$ 2.1% conversion.
- Treatment (B): 2,000 users $\rightarrow$ 2.5% conversion (+0.4% lift).
- Product Manager wants to ship immediately!
- **Socratic Probe:** Is this +0.4% lift a real behavioral change or just random coin-flip variance?

The "Aha!" Discovery: Proof by Contradiction

- **Null Hypothesis (H_0):** Treatment has **zero effect** ($\mu_B - \mu_A = 0$). Presumption of innocence!
- **Simulate the Null Universe:** What is the distribution of differences if Variant B is completely useless?
- **p-Value:** How rarely would random noise generate a lift $\geq +0.4\%$?

Important p-value measures surprise under the null hypothesis.

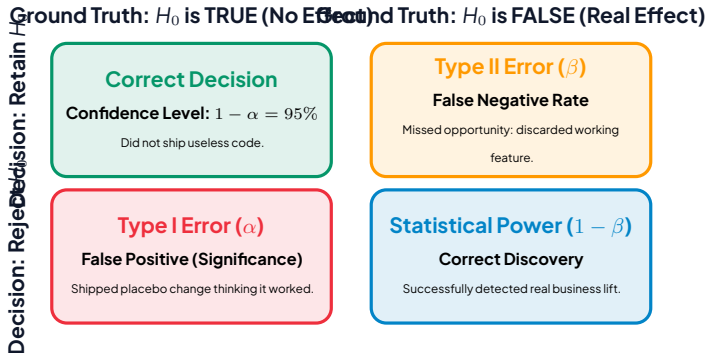
Hero Visualization: Null Distribution & Rejection Region



Important

Reject H_0 if observed z^* falls into the critical $\alpha = 0.05$ extreme tail.

Hero Visualization: Type I vs Type II Decision Matrix



Production Checkpoint: Non-Parametric Permutation Testing

Permutation Test Algorithm

1. Compute observed metric delta $\Delta^* = \bar{B} - \bar{A}$.
2. Pool all observations ($N_A + N_B$).
3. Reshuffle group labels $K = 10,000$ times.
4. Fraction of shuffled trials with $\Delta_{\text{null}} \geq \Delta^*$ gives the exact empirical p -value.

Important Permutation tests eliminate all parametric assumptions.

NumPy Implementation

```
1 import numpy as np
2
3 def permutation_test(ctrl, trt, k=10_000):
4     obs_diff = np.mean(trt) - np.mean(ctrl)
5     pooled = np.concatenate([ctrl, trt])
6     n = len(ctrl)
7     null_diffs = np.empty(k)
8     for i in range(k):
9         np.random.shuffle(pooled)
10        null_diffs[i] = np.mean(pooled[n:]) - np.mean(pooled[:n])
11    return float(np.mean(null_diffs >= obs_diff))
```

Session Summary

01 The Illusion of Certainty: Bayes' Theorem

Rare events, base rate fallacy, conditional probability geometry, and expectation.

02 Order from Chaos: The Central Limit Theorem

Why bell curves rule the universe, Law of Large Numbers, and standard errors.

03 Hypothesis Testing & Decision Making

The skeptic's null hypothesis, p-values, Type I/II errors, and statistical power.

Professional Program in Enterprise Data Science & Agentic AI with AWS — Modular Session Architecture